Firmware Reference

Main ROM IC519: Disassembly, Command Set and Data Formats

Reverse-Engineering Supplement to the Operation and Service Manuals
Document VS-2724A-90F01
Derived from ROM images 2724ASTD.bin and 2724ACPR.bin
Jared Cabot • jetstreamtechnology@protonmail.com
Firmware analysis and text prepared with Claude (Anthropic)
Revision 1.2 • October 2026

Scope

This supplement documents the firmware in IC519, the 8K x 8 main ROM of the Valhalla Scientific Model 2724A and 2724CPR Programmable Precision Resistance Standard. It covers the processor and memory map, the display and keyboard, the IEEE-488 command interpreter, the non-volatile data format and the calibration sequence.

It records what the firmware does, and in particular the behaviour that the Operation Manual and Service Manual do not describe. Where this document and the factory manuals disagree, it says so.

This is not a repair manual. Circuit descriptions, adjustment procedures and parts lists are in the Service Manual, which this document supplements and does not replace. The Option TL-1 interface board is covered only as far as IC519 drives it.

Provenance

The firmware description comes from static analysis of two 8192-byte ROM images, 2724ASTD.bin and 2724ACPR.bin, disassembled for the Motorola 6809 and traced from their hardware vectors. Both images pass the firmware's own power-on ROM test, and all seven live interrupt vectors in both point into the ROM.

Hardware facts (the processor, the memory devices, the interrupt assignments and the strap wiring) are taken from the Service Manual and from drawings 2724-070 and 2724-071, and are cited where used. Everything else is read from the code.

The CPR image has since been programmed into a replacement EPROM and runs correctly in the author's instrument. The undocumented commands have not yet been tried on it. Chapter 11 records what was and was not checked.

Applicability

Two builds are described: the standard build, Valhalla part 03-50019, ROM checksum $F688; and the CPR build, part 03-50126, checksum $A4B9. The checksum is the word at $E000 and is the quickest way to identify a device.

Only 15.2% of the bytes are the same at the same address in the two builds, so any address that differs between them is given for both. On a device whose checksum matches neither value, treat every address here as unverified.

Conventions

Hexadecimal values carry a $ prefix, addresses and register contents alike; binary values carry a % prefix. Addresses are CPU addresses in the 6809's 64K space. An operand written `<$nn` is a direct-page address, $C0nn.

The monospaced face is used for text sent to or from the instrument, text shown on its display, and instructions and symbols taken from the firmware. Angle brackets enclose a placeholder: `<n>` is a single decimal digit.

Documented means the behaviour appears in the 2724A Operation Manual or Service Manual; undocumented means it appears in neither. Chapter and table numbers refer to this document unless a factory manual is named.

Preparation

Jared Cabot supplied the ROM images, the factory manuals and drawings, and the results from the instrument. The disassembly, the analysis and the text were prepared with Claude (Anthropic). Key names, quoted manual wording and the section and table numbers of the factory manuals are drawn from the Valhalla Scientific Operation and Service Manuals; the rest is the result of this analysis.

Disclaimer

This is not a Valhalla Scientific publication and has no endorsement from that company or its successors. It was produced to maintain and repair obsolete test equipment.

Several undocumented commands write to non-volatile memory or drive the calibration sequence. Used on a working, calibrated instrument they can destroy its calibration. Read Chapter 9 before trying any of them.

Contents

1. General Information **1-1**
The Instrument and Its Firmware 1-1
Identifying a ROM 1-1
Summary of Undocumented Findings 1-2

2. Processor and Memory **2-1**
Processor 2-1
Interrupt Vectors 2-1
Memory Map 2-2
IEEE-488 Controller Registers 2-2
Instrument I/O Ports 2-3
Read/Write Memory Layout 2-4

3. Display and Keyboard **3-1**
Character Set 3-1
Message Format 3-1
Keyboard Scanning 3-2
Key Matrix Map 3-2
Matrix Differences in the CPR Build 3-3
Greeting Selection 3-3
The Other Straps and Switches 3-4

4. Message Catalogue **4-1**
Messages Present in Both Builds 4-1
Messages Absent from the Manuals 4-2
Differences from the Operation Manual's Message List 4-3

5. Remote Operation **5-1**
Interface Hardware 5-1
Character Reception and the Input Buffer 5-1
The Command Interpreter 5-2
Unrecognised Commands 5-2

6. Programming Reference **6-1**
The Complete Command Set 6-1
Argument Ranges Enforced by the Firmware 6-2
Numeric Entry 6-2
Commands Absent from the Manuals 6-2

Contents (cont'd)

7. Command Reference **7-1**
Introduction 7-1
Notation 7-1
Command Summary 7-2
Command Entries 7-2

8. Non-Volatile Data **8-1**
NOVRAM Organisation 8-1
Block Layout and Checksums 8-1
Store and Recall 8-1
Corruption Behaviour 8-2

9. Calibration and Integrity Checks **9-1**
The Calibration Step Counter 9-1
Driving Calibration Remotely 9-1
The Banner Integrity Check 9-2

10. Firmware Differences **10-1**
Common Regions 10-1
Functional Differences 10-2

11. Verification Record **11-1**
Checks Performed 11-1
Not Verified 11-2
How to Reproduce 11-2

1

General Information

The Instrument and Its Firmware

The Valhalla Scientific Model 2724A is a programmable precision resistance standard. It has no switched resistor network. It synthesises a resistance actively: it measures the current forced through an internal reference resistor and drives its output terminals to the voltage that a resistor of the requested value would develop. The Operation Manual calls this the active ohm technique. Option CPR adds a set of fixed cardinal-point resistors and a second operating mode, CPR mode.

Everything above the analogue circuits is done in software. A single 8K x 8 EPROM at IC519 holds the display and keyboard scanner, the IEEE-488 command interpreter, the calibration sequence and the non-volatile data manager. There are two builds of that ROM, one for the standard instrument and one for the CPR option.

The Service Manual gives the processor as an MC68A09P at IC518 and describes the memory in Section 9-3-6-5. It does not document the firmware; this supplement does.

Chapters 2 and 3 describe the hardware as the firmware sees it, and Chapter 4 lists every display message. Chapters 5 to 7 cover remote operation and end with a reference entry for each bus command. Chapters 8 and 9 cover the non-volatile data and the calibration sequence, Chapter 10 compares the two builds, and Chapter 11 records how each finding was checked.

Identifying a ROM

At every power-on the firmware sums the 8190 bytes from $E002 to $FFFF as 16-bit words and compares the result with the word stored at $E000. That stored word identifies the build, and it can be read straight from a device image without any disassembly.

Table 1-1. ROM Build Identification

Build	Valhalla part	Checksum at $E000	Size	Image analysed
Standard	03-50019	$F688	8192 bytes	2724ASTD.bin
CPR	03-50126	$A4B9	8192 bytes	2724ACPR.bin

Note



Both images analysed pass their own checksum test. A device whose sum does not match the word at $E000 stops at reset with all display decimal points lit, which Table 3-2 of the Operation Manual lists as ROM failure.

Summary of Undocumented Findings

The following are present in both builds and appear in neither factory manual. The right-hand column gives the chapters that describe each one.

Table 1-2. Undocumented Behaviour at a Glance

Finding	Chapters
Bus command `SM<n>` stores the present value in user memory	6, 7
Bus command `RM<n>` recalls a user memory	6, 7
Bus command `RL` recalls the last entered value	6, 7
Bus command `N` advances the calibration sequence	6, 7, 9
Six punctuation and control characters are ignored in a command line	5
Message `IEEE NOT FITTED`	4, 5
Messages `CAL ABORTED`, `FAST DATA OK`, `NO FAST DATA` and `UNDERCURRENT`	4
Prompts `STORE MEMORY`, `RECALL MEMORY` and the six `SEL` mode prompts	4
The A-to-D fault reads `A-D TIMEOUT`, not `ATOD TIMEOUT` as in the manual	4
The assignments of the instrument's I/O ports	2
A banner-comparison integrity check with an obscene failure message	9

2

Processor and Memory

Processor

IC518 is an MC68A09P, the 1.5 MHz grade of the Motorola 6809. The Service Manual parts list names the part, and the code confirms it: it uses instructions and addressing modes found on no other 8-bit Motorola processor, such as LEAX and LEAY with program-counter-relative offsets, the U stack, the TFR and EXG register pairs, and the FIRQ interrupt.

The reset routine loads the direct page register with $C0 and never changes it, so every direct-page operand addresses $C000 to $C0FF. This document writes such operands as <$nn, as the assembler does, and gives the full address where it matters.

Interrupt Vectors

Section 9-3-6-3 of the Service Manual assigns the four interrupts. The vectors in both builds agree with it.

Table 2-1. Interrupt and Reset Vectors

Vector	Name	Standard	CPR	Service
$FFF0	(reserved)	$0000	$0000	Unused
$FFF2	SWI3	$EBB0	$EE29	Internal
$FFF4	SWI2	$EB85	$EDFE	Internal
$FFF6	FIRQ	$E860	$EAAF	Display and keyboard scan: next digit due
$FFF8	IRQ	$EFF1	$F2B5	IEEE-488 bus activity
$FFFA	SWI	$EF2C	$F1F0	Command line complete: run the interpreter
$FFFC	NMI	$EBE3	$EE5C	New A-to-D result available
$FFFE	RESET	$F9D8	$FD51	Power-on and remote reset

Note



In this design the SWI vector is not an error trap. When the IEEE-488 receive interrupt has buffered a complete command line it executes SWI, and the interpreter runs in the software interrupt handler. See Chapter 5.

Memory Map

The address decoder uses only A9 to A15, so devices are selected in 512-byte blocks and each one repeats through its block. The table gives the addresses the firmware uses.

Table 2-2. Memory Map

Address range	Device	Size	Notes
$0000 - $0FFF	Output latches IC512-IC517 and IC526; keyboard input	-	Selected in 512-byte blocks; see Table 2-4
$1000 - $1007	IEEE-488 interface (Option TL-1)	8 bytes	Absent when the option is not fitted
$13FF	A-to-D converter	2 bytes	Read by the NMI handler
$2000	NOVRAM STORE strobe	-	A read copies the NOVRAMs' internal RAM into their non-volatile array
$4000	NOVRAM RECALL strobe	-	A read copies the non-volatile array back into internal RAM
$A000 - $A0B3	NOVRAM (IC522 and IC523)	180 x 8 bits	IC522 supplies D0-D3, IC523 supplies D4-D7
$C000 - $C3FF	Static RAM (IC520, IC521)	1024 bytes	Direct page is $C000-$C0FF
$E000 - $FFFF	Main ROM (IC519)	8192 bytes	Checksum word at $E000; vectors from $FFF0

IEEE-488 Controller Registers

The IEEE-488 controller occupies eight consecutive addresses. The parts list does not name the device, so the register names below follow the usual talker/listener controller layout, not a datasheet.

Table 2-3. IEEE-488 Controller Registers as Used by the Firmware

Address	Access	Firmware use
$1000	R/W	Interrupt status; also written with the enable copy held at `<$59`
$1001	R	Secondary status: bus command and addressing events
$1002	W	Written once during initialisation
$1003	W	Auxiliary command register; $20 and $10 written in the receive path
$1004	R/W	Address register
$1005	R/W	Serial poll status byte
$1006	W	Parallel poll response
$1007	R/W	Data in and data out

Instrument I/O Ports

Within the latch block of Table 2-2 the firmware uses the addresses below. Their purpose comes from the code that uses them; none of these assignments appears in the manuals.

Table 2-4. Input and Output Ports Used by the Firmware

Address	Access	Width	Function
$0000	R	8	Keyboard return lines K1 to K4 on bits 2 to 5 (Chapter 3). Bit 7 reads 0 at the end of each display scan
$0200	W	8	Cleared once during reset
$03FF	W	16	Display segment and digit drive, written by the FIRQ handler
$0400	W	8	Written with $40 once during reset
$07FF	W	16	Main relay and range data, written with interrupts masked
$0A00	W	8	Further relay data, written straight after $07FF with interrupts still masked
$0C00	W	8	Mode latch. Bit 7 is set in active mode and clear in CPR mode
$0E00	W	8	Mode latch. Bit 7 is set in slow mode and clear in fast mode
$13FF	R	16	A-to-D converter result, read by the NMI handler

A width of 16 means the firmware reaches the address with a 16-bit instruction, `STD` or `LDD`. Because decoding is in 512-byte blocks, a 16-bit access at $03FF, $07FF or $13FF falls on a block boundary: its first byte goes to the block that ends at that address and its second to the block that starts at the next, so one instruction reaches two 8-bit devices. This follows from the address decoding and has not been checked on the circuit.

The relay data at $07FF and $0A00 is always written with interrupts masked by `ORCC #$50`, so the relay drive is never left half-updated. Both mode latches have bit 7 set in the power-up state, so an instrument that has not been programmed is in active mode and slow mode.

Read/Write Memory Layout

The reset routine tests all 1024 bytes of RAM with three patterns before using it. The table lists the allocations the code identifies with confidence. The rest of the direct page holds working variables whose purpose was not established.

Table 2-5. Identified RAM Allocations

Address	Bytes	Contents
$C012	1	Calibration step counter; zero when not calibrating
$C017	1	Strap and switch inputs, read at keyboard scan position 8
$C024	32	Display buffer, one 16-bit word per digit
$C04F	1	Service request mask, set by `Q`
$C050	1	Output delimiter code, set by `E`
$C051	1	Parallel poll response line, set by `P`
$C052	1	Two- or four-wire selection, set by `T`
$C053	1	Fast or slow selection, set by `M`
$C054	1	CPR or active selection, set by `C` (CPR build only)
$C05A	2	Input buffer write pointer
$C05D	2	Interpreter read pointer
$C066	1	Memory number for `SM` and `RM`
$C069	8	Present output value
$C071	8	Value being entered
$C134	100	IEEE-488 input buffer, $C134 to $C197
$C1CA	8	Last entered value, for `RL`
$C23F	-	Stack top
$C300	88	Calibration data
$C358	2	Calibration data checksum
$C35A	2	User configuration, including the IEEE-488 address
$C35C	80	Ten user memories of 8 bytes each
$C3AC	2	User data checksum

3

Display and Keyboard

Character Set

The display is a 16-digit, 14-segment vacuum fluorescent panel. The main board steps through the digits one at a time, and each step raises a FIRQ in which the firmware writes the next digit.

The firmware does not store text as ASCII. Each character is stored as an index into a 43-entry table at $E058, which gives the ASCII code that the display's segment decoder expects. The table is identical in both builds. In Table 3-1 the index of a character is the row value plus the column value.

Table 3-1. Display Character Table at $E058

Index	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A
$00	0	1	2	3	4	5	6	7	8	9	*
$0B	A	C	D	E	*sp*	B	F	G	H	I	J
$16	K	L	M	N	O	P	Q	R	S	T	U
$21	V	W	X	Y	Z	-	/	o	u	m	

Indices $00 to $09 are the digits, so a numeric value is displayed without translation. Index $0F is the space. The letters are not in alphabetical order (B comes after the space), which suggests the table grew as messages were written. Only 43 of the 256 possible codes are defined; a message byte above $2A has no entry and displays unpredictably.

Message Format

Every message is exactly 15 bytes of character-table indices, with no terminator and no length byte. Messages are packed end to end in blocks, and a message is found by multiplying its number within the block by 15 and adding the block base. The code does this directly:

```
LEAX  block,PCR      ; base of the message block
DECA                 ; message number, 1-based
LDB   #$0F           ; 15 bytes per message
MUL
ABX                  ; X = block + (n-1) * 15
LBSR  display        ; $E9E6 standard, $EC55 CPR
```

The display routine takes the pointer in X and writes 15 characters. The sixteenth digit is not part of the message; it shows annunciator state.

Keyboard Scanning

The same FIRQ handler scans the display and the keyboard. Nine of the lines that select display digits also leave the main board as scan lines: eight drive the keyboard and the ninth drives the greeting straps. Drawing 2724-071 calls these the keyboard column drives. Four return lines, K1 to K4, come back from the keyboard. The firmware reads them at $0000 on bits 2 to 5, and a pressed key reads as 0.

Each FIRQ writes the next display word to $03FF, reads $0000 and counts one scan position, so the return lines are sampled against whichever scan line is driven at that moment. A read with bit 7 clear marks the end of a display scan and restarts the count.

A pressed key is latched as a return mask at `<$1F` and a scan position at `<$20`. The handler compares each new sample with these two bytes and acts only on a change, which debounces the key. Both are cleared on release, so a key held down produces one action.

The decoder at $F77D (CPR; $F432 standard) turns the latched pair into a key number. It shifts the return mask right twice, finds the lowest set bit, and forms an index of four times the scan position, less one, plus that bit number. If more than one bit is set it returns $FF and the press is discarded, so two keys pressed together are ignored rather than resolved.

The index selects a two-byte entry in the key matrix table at $F79C (CPR; $F43E standard). The first byte is the key number, which selects a handler from the key dispatch table ($F385 CPR, $F0C1 standard). The second is a sub-code left at `<$79` for that handler to read. This is how one handler serves several keys.

Key Matrix Map

Positions 1 to 7 and 9 are keyboard positions; position 8 is reserved for the straps and is never decoded as a key. Each cell gives the key number and, where it matters, the sub-code after a point: 5.9 is key 5 with sub-code 9. A dash marks a position with no key.

Table 3-2. Key Matrix, Standard Build

Scan position	K1 (bit 2)	K2 (bit 3)	K3 (bit 4)	K4 (bit 5)
1	8	9	10	- (see note)
2	12	11.2	11.1	11.0
3	5.9	5.6	5.3	6
4	5.8	5.5	5.2	5.0
5	5.7	5.4	5.1	7
6	4	3	1	2
7	-	-	-	0
8	*straps*	*straps*	*straps*	*straps*
9	14.1	14	13	13.1

Key 5 is numeric entry, and its sub-code is the digit, which is why positions 3 to 5 read as a keypad: 9 6 3, 8 5 2 0, 7 4 1. Key 11 appears three times, with sub-codes 2, 1 and 0. These are the three unit keys, and key 11 is also the handler that the bus command `N` reaches, which is what lets calibration be driven over the bus (Chapter 9).

Note



Position 1, K4 has no table entry. The FIRQ handler tests for that key before it consults the table and handles it in the IEEE-488 code. It is the MAN key, which the Operation Manual says stays active in remote. Every other key is ignored while the instrument is in remote.

Matrix Differences in the CPR Build

Positions 1 to 8 are the same in both builds. Position 9 differs, and it holds the two extra keys of the CPR instrument.

Table 3-3. Scan Position 9 in Both Builds

Build	K1 (bit 2)	K2 (bit 3)	K3 (bit 4)	K4 (bit 5)
Standard	14.1	14	13	13.1
CPR	14.1	16	15	13.1

Key 15 is CARD PNT. It toggles the byte at `<$54` and shows `SEL CPR MODE` or `SEL ACTIVE`. The bus command `C` writes the same byte, so the key and the command change one setting. Key 16 matches PPM DEV, which the Operation Manual says works only in CPR mode: the firmware refuses it unless `<$54` is set, and otherwise complements `<$7A`, the flag that lights the key's LED.

The standard build maps these two positions to keys 14 and 13, the same keys as their neighbours, so a standard instrument has no dead positions where the CPR keys would be.

Greeting Selection

The Operation Manual notes that instruments shipped outside the United States may greet the operator in another language, but does not say how the language is chosen. Note 4 on drawing 2724-071, the display and keyboard schematic, gives the selection directly. The choice is made by diode straps, not by a firmware variant, and both builds contain all four greetings.

Table 3-4. Greeting Selection, from Drawing 2724-071 Note 4

Diodes fitted	Note 4 text	$C017 bits 3, 2	Index	Firmware text
None	HI-THERE	%11	0	`HI-THERE`
D701	HELLO	%10	1	`HELLO`
D702	GUTEN TAG	%01	2	`GUTEN TAG`
All	BON JOUR	%00	3	`BONJOUR`

D701 to D704 are four optional 1N4148 diodes on the display board. Their cathodes are joined to the strap scan line from SK2-14, and their anodes go to the return lines K1 to K4. Each return line is held up by a 1 k resistor (R703 to R706) to +5 V(E) and leaves the board on SK2-57 to SK2-60. A fitted diode pulls its line low when the strap scan line is driven. Because the straps use the keyboard return lines, they need no extra wiring.

The firmware reads the return lines at scan position 8 and stores the result at $C017. It forms the greeting index from bits 3 and 2:

```
LDA   <$17            ; strap and switch inputs
ORA   #$F3            ; keep bits 3 and 2
COMA                  ; invert: a fitted diode reads as 1
LSRA
LSRA                  ; index 0 to 3
LDB   #$0F
MUL
ABX                   ; X = greeting block + index * 15
```

The `COMA` is what makes the table read as it does. A position without a diode reads high, so no diodes gives index 0 and the first greeting. Working the arithmetic through for all four combinations reproduces note 4 exactly, which confirms both the reading of the code and the bit assignment: D701 is on bit 2 and D702 on bit 3.

Note



Note 4 writes the fourth greeting as two words, BON JOUR; the firmware stores `BONJOUR`. The Operation Manual writes the first greeting HI THERE, without the hyphen that the firmware and note 4 both use.

The Other Straps and Switches

Note 4 gives no function for D703 and D704. They sit on K3 and K4, which the keyboard scan reads on bits 4 and 5, and neither build ever tests bit 4 or bit 5 of $C017. In both builds, therefore, D703 and D704 have no effect.

The firmware does test three other bits of $C017. None of them comes from the keyboard return lines; they are further inputs read at the same time.

- Bit 6 is the rear calibration keyswitch, which reads 0 when the switch is on. The main loop starts calibration when it sees bit 6 clear, and the end of calibration waits for it to be set again (Chapter 9).
- Bit 1, when set, sets the flag at $C089 on each pass of the main loop.
- Bit 0, when set, makes the main loop clear the output-enable flag at $C084, refresh the output latches and return to the top of the loop without running the normal output path. That is the behaviour of an output inhibit.

Where bits 0 and 1 come from was not traced. The keyboard input buffer is on main board drawing 2724-070 sheet 5, and its wiring would settle it.

4

Message Catalogue

Messages Present in Both Builds

The catalogue was produced by decoding every run of valid character-table indices in both images through the table at $E058 and keeping the runs that form 15-character messages. Addresses are given for both builds. The text is reproduced exactly, including the spaces that centre each message on the display.

Table 4-1. Fault and Status Messages

Message	Std	CPR	Meaning	In manuals
`TURN CAL SW OFF`	$E083	$E083	Calibration switch still on at the end of a calibration	Yes
`CAL DATA OK`	$E092	$E092	Power-on self-test result	Yes
`CAL ABORTED`	$E3C9	$E503	Calibration sequence abandoned	No
`FAST DATA OK`	$E3D8	$E512	Fast-mode calibration data accepted	No
`NO FAST DATA`	$E3E7	$E521	No fast-mode calibration data present	No
`FUCK FUCK FUCK`	$E405	$E53F	Banner integrity check failed (Chapter 9)	No
`CLOCK FAILURE`	$EAFE	$ED77	Faulty clock circuit	Yes
`MATH OVERFLOW`	$EB0D	$ED86	Arithmetic overflow	Yes
`CAL DATA BAD`	$EB1C	$ED95	Calibration data corrupted	Yes
`MEMORY DATA BAD`	$EB2B	$EDA4	User memory data corrupted	Yes
`OUT OF CAL`	$EB3A	$EDB3	A component is out of tolerance	Yes
`SETTLING`	$EB49	$EDC2	Output has not yet settled	Yes
`CANNOT CONTROL`	$EB58	$EDD1	Output cannot reach the requested value	Yes
`IEEE NOT FITTED`	$EB67	$EDE0	IEEE-488 option not installed	No
`A-D TIMEOUT`	$EB76	$EDEF	A-to-D conversion timed out	Spelled differently

Table 4-2. Prompt and Greeting Messages

Message	Std	CPR	Meaning	In manuals
STORE MEMORY	$F25A	$F535	Waiting for a memory number to store	No
RECALL MEMORY	$F29D	$F57A	Waiting for a memory number to recall	No
UNDERCURRENT	$F356	$F668	Test current below the usable minimum	No
SEL SLOW MODE	$F38A	$F69C	Slow mode selected	No
SEL FAST MODE	$F399	$F6AB	Fast mode selected	No
SEL FOUR-WIRE	$F3A8	$F6BA	Four-wire selected	No
SEL TWO-WIRE	$F3B7	$F6C9	Two-wire selected	No
SEL CPR MODE	$F3C6	$F6D8	CPR mode selected	No
SEL ACTIVE	$F3D5	$F6E7	Active mode selected	No
HI-THERE	$F982	$FCFB	Power-on greeting, chosen by straps (Chapter 3)	Yes
HELLO	$F991	$FD0A	Power-on greeting, chosen by straps	Yes
GUTEN TAG	$F9A0	$FD19	Power-on greeting, chosen by straps	Yes
BONJOUR	$F9AF	$FD28	Power-on greeting, chosen by straps	Yes

Messages Absent from the Manuals

Fourteen of the twenty-eight messages appear in neither factory manual. Ten of them can appear in ordinary use:

- IEEE NOT FITTED: shown when a bus function is attempted on an instrument without Option TL-1. Table 3-2 of the Operation Manual does not list it.
- UNDERCURRENT: the counterpart of the overcurrent indication. Section 4.5 of the Operation Manual describes the U status character that marks undercurrent in the bus output, but does not list the display message.
- STORE MEMORY and RECALL MEMORY: prompts shown while the instrument waits for a memory digit.
- The six SEL prompts, shown as each mode is selected.

The other four, CAL ABORTED, FAST DATA OK, NO FAST DATA and the banner-check message, appear only during calibration or after a failed integrity check (Chapter 9).

Differences from the Operation Manual's Message List

A technician comparing the display with Table 3-2 of the Operation Manual will meet three differences.

Table 4-3. Firmware Text Against Manual Text

Operation Manual, Table 3-2	Firmware	Comment
ATOD TIMEOUT	`A-D TIMEOUT`	Different spelling, same fault
HI THERE	`HI-THERE`	The firmware greeting is hyphenated
-	`IEEE NOT FITTED`	Message absent from the manual

Note



Table 3-2 introduces its list with the words "Some of the messages", so it does not claim to be complete. It is still the only list a user has, and it leaves out a message that an instrument without the IEEE-488 option can show.

5

Remote Operation

Interface Hardware

The IEEE-488 interface is Option TL-1, on a separate board, 2724-602. The main ROM contains the whole command interpreter whether or not the option is fitted; an instrument without it lacks only the eight interface registers at $1000. The firmware detects this and can show `IEEE NOT FITTED`, a message that appears in neither manual.

Character Reception and the Input Buffer

The IRQ handler receives each character. It reads the byte from $1007, clears bit 7 and appends the result to a 100-byte buffer from $C134 to $C197. Nothing is interpreted at this stage. A line ends on any of three conditions:

- EOI asserted with the character, seen in the status register.
- A carriage return, $0D.
- The buffer filling up at $C198.

The handler then disables reception, writes a zero after the last character and executes `SWI`. The interpreter therefore runs in the software interrupt handler, not inside the receive interrupt.

The receive handler also drops every space ($20) before it reaches the buffer, so spaces may appear anywhere in a command line, even in the middle of a number.

Note



Line feed does not end a line, which agrees with Section 4.3 of the Operation Manual: it lists only CR and EOI as input delimiters. A controller that ends its output with CR LF leaves the LF at the start of the next line, where the interpreter ignores it. A controller that sends LF alone never ends a line, and the instrument keeps accepting characters until the buffer is full at 100 bytes.

The Command Interpreter

The interpreter is a simple table walk. It takes the next character from the buffer and searches a dispatch table of four-byte entries: one ASCII character followed by a three-byte long branch to its handler. A zero byte ends the table. On a match the interpreter jumps into the entry itself, so the long branch does the dispatch.

```
entry:   FCB   'x'          ; the command character
         LBRA  handler      ; three bytes, executed in place
...
         FCB   $00          ; end of table
```

If the character is not in the dispatch table, the interpreter searches a second, much shorter table. A character found there is ignored and the interpreter moves on, which is why punctuation can appear in a command line without causing an error.

Table 5-1. Characters Ignored Without Error

Code	Character	Name
$2C	,	comma
$3B	;	semicolon
$3A	:	colon
$2D	-	minus
$2B	+	plus
$0A	-	line feed

Note



The plus and minus signs are in this list, so a sign at the top level of a command line is ignored, not applied. Signs are recognised only inside an exponent, where the numeric parser handles them itself. The list is identical in both builds.

Unrecognised Commands

A character found in neither table is an error. The firmware tests bit 1 of the service request mask at $C04F, which the `Q` command sets. If service requests on input data errors are enabled, it loads status code $18 and asserts SRQ. Either way, the rest of the line is discarded.

This matches the Operation Manual. Section 4.3 says that decoding stops at an invalid character and the rest of the buffer is discarded, and the description of `Q2` gives an undecodable command as its example of an input data error.

6

Programming Reference

The Complete Command Set

Table 6-1 is taken from the dispatch tables in the two ROM images, in table order, so it is complete: a character not listed is not a command. Counting the ten digits, the standard build recognises 23 characters and the CPR build 24. Handler addresses for each build are in Table 7-1.

Table 6-1. Command Characters in Dispatch Order

Char	Function	Builds	In manuals
`U`	Step control: up	Both	Yes
`D`	Step control: down; also starts `DON` and `DOFF`	Both	Yes
`L`	Step control: left	Both	Yes
`R`	Step control: right; also starts `RM` and `RL`	Both	Bare `R` only
`S`	Starts `SM`, store to user memory	Both	No
`A`	Reset, the same as Device Clear	Both	Yes
`Q`	Service request mask	Both	Yes
`E`	Output delimiter	Both	Yes
`C`	CPR or active mode	CPR only	Yes
`M`	Fast or slow mode	Both	Yes
`T`	Two- or four-wire	Both	Yes
`P`	Parallel poll response	Both	Yes
`N`	Advance the calibration sequence	Both	No
`.`	Decimal point in numeric entry	Both	Yes
`0` to `9`	Numeric entry	Both	Yes

Note



Command characters are matched as upper case only. Nothing in the receive or dispatch path folds case, so `m1` is not `M1`. The one exception is the exponent character inside a number, where the numeric parser accepts both `E` and `e`, as the Operation Manual states.

Argument Ranges Enforced by the Firmware

Each command that takes a digit checks its range and treats a digit outside it as an undecodable command, which discards the rest of the line. The limits come from the comparison instructions in each handler.

Table 6-2. Enforced Argument Ranges

Command	Accepted	Stored at	Operation Manual
Q<n>	0 to 7	$C04F	Q0 to Q7: agrees
E<n>	0 to 4	$C050	E0 to E4: agrees
P<n>	0 to 8	$C051	Range not stated
T<n>	0 to 1	$C052	T0, T1: agrees
M<n>	0 to 1	$C053	M0, M1: agrees
C<n>	0 to 1	$C054	C0, C1: agrees
SM<n>	0 to 9	$C066	Command not documented
RM<n>	0 to 9	$C066	Command not documented

P accepts nine values. That fits the manual's description, in which P0 selects no response and P1 to P8 select a DIO line, but the manual never states the upper limit. P9 is rejected.

Numeric Entry

A digit or a decimal point starts numeric entry. It runs its own loop outside the dispatch table until it meets a character it does not accept; that character is pushed back and the interpreter carries on from it. Numeric entry accepts digits, one decimal point, and an exponent introduced by E or e, with an optional sign.

The exponent parser builds the exponent in one byte at $C055 and records a minus sign at $C056. It keeps only the last pair of digits it forms, so the exponent is not range-checked the way the value itself is.

Commands Absent from the Manuals

Four command forms exist in both builds and appear in neither manual. Each has a front-panel equivalent, and the same handler serves the key and the bus command, which is the likely reason they exist. Chapter 7 has a full entry for each.

Table 6-3. Undocumented Command Forms

Form	Effect	Front-panel equivalent
SM<n>	Store the present value in user memory <n>	STO MEM, then a digit
RM<n>	Recall user memory <n> into the present value	RCL MEM, then a digit
RL	Recall the last entered value	RCL LAST
N	Advance the calibration sequence one step	A unit key, during calibration

Caution



SM<n> writes to non-volatile memory. N drives the calibration sequence and, at the end, writes calibration constants to NOVRAM. Do not send either to a calibrated instrument without reading Chapters 8 and 9.

7

Command Reference

Introduction

This chapter has one entry for each command the dispatch table accepts. Chapter 6 describes the command set as a whole and the ranges the firmware enforces; this chapter is the per-command reference.

The interpreter dispatches on a single character and has no subsystem structure, so the entries are in alphabetical order, the order of Table 7-1. Every command applies to both builds unless its entry says otherwise.

Notation

Each entry opens with the command character, followed by one line for each accepted form. <n> is a single decimal digit; its permitted range is given in the text and enforced by the firmware. Letters are matched exactly as shown, in upper case only.

The last paragraph of each entry gives the handler address in each build, followed by a qualifier in parentheses:

- **(No response)** The command produces no reply. The 2724A has no query commands: its output buffer always holds the display contents, as Section 4.3 of the Operation Manual describes.
- **(Not documented)** The command works but appears in neither factory manual. It produces no reply either.
- **(CPR build only)** The standard build has no dispatch entry for the command and treats it as undecodable.

Command Summary

The handler addresses are the addresses the dispatch table branches to, computed from the 16-bit offsets the table stores.

Table 7-1. Dispatch Table Contents

Command	Standard handler	CPR handler	Operation Manual
A	$F9CD	$FD46	Documented
C	absent	$F035	Documented, CPR only
D	$ECAD	$EF56	Documented
E	$ED5E	$F007	Documented
L	$ECE3	$EF8C	Documented
M	$ED75	$F01E	Documented
N	$EDBA	$F07A	Not documented
P	$EDA3	$F063	Documented
Q	$ED47	$EFF0	Documented
R	$ECE9	$EF92	Bare R documented
S	$ED25	$EFCE	Not documented
T	$ED8C	$F04C	Documented
U	$ECA5	$EF4E	Documented
.	$EDCF	$F08F	Documented
0 to 9	$EDE8	$F0A8	Documented

Command Entries

A

A

Resets the instrument, like an IEEE-488 Device Clear. The handler is the reset entry point, so the full power-on sequence runs, including the ROM and RAM tests and the greeting.

The Operation Manual says the command is for controllers without a Device Clear capability.

Handler $F9CD standard, $FD46 CPR. **(No response)**

C

`C0`

`C1`

`C0` selects active mode, the power-up condition. `C1` selects CPR mode. The digit is stored at $C054, the byte that the CPR front panel's CARD PNT key toggles (Chapter 3).

The standard build has no `C` entry, so a standard instrument treats `C1` as an undecodable command and discards the rest of the line.

Handler $F035, CPR build. **(CPR build only)**

D

`D`

`DON`

`DOFF`

Bare `D` is the step-control down arrow. `DON` turns step control on and `DOFF` turns it off.

The handler looks at the next character without consuming it. If it is `O`, the longer forms are parsed; otherwise the bare arrow action is taken. Any other letter after `DO` is an error, as is `DOF` with one F.

Handler $ECAD standard, $EF56 CPR. **(No response)**

E

`E<n>`

Sets the output delimiter; Section 4.5 of the Operation Manual lists the five codes. The digit is checked for 0 to 4 and stored at $C050. Any other digit is an undecodable-command error.

`E` is also the exponent character inside a number. The two uses do not conflict, because the numeric parser reads the exponent itself and does not return to the dispatch table until the number is complete.

Handler $ED5E standard, $F007 CPR. **(No response)**

L

`L`

Step-control left arrow: moves the active digit one place towards the most significant end.

Handler $ECE3 standard, $EF8C CPR. **(No response)**

M

`M0`

`M1`

`M0` selects slow mode, the power-up condition. `M1` selects fast mode. The digit is checked for 0 or 1 and stored at $C053, from which bit 7 of the latch at $0E00 is set.

Handler $ED75 standard, $F01E CPR. **(No response)**

N

`N`

Advances the calibration sequence by one step, accepting the value entered since the previous step. It is the bus equivalent of pressing a unit key during the front-panel calibration of Section 6.3.4 of the Operation Manual: the three unit keys and this command share one handler, key 11 of the key dispatch table.

The handler first tests the calibration step counter at $C012 and returns at once if it is zero, so outside calibration `N` has no effect.

During calibration it stores the entered value, increments the step counter and moves to the next step. When the counter reaches ten the sequence ends and the constants are written to NOVRAM.

Handler $EDBA standard, $F07A CPR; both continue at the unit-key handler, $F30E standard, $F5EF CPR. **(Not documented)**

Caution



Sending `N` to an instrument with its rear calibration keyswitch on advances any calibration in progress and can write wrong constants to non-volatile memory. See Chapter 9.

P

`P<n>`

Sets the parallel poll response. `P0` selects no response; `P1` to `P8` select the DIO line the instrument asserts. The digit is checked for 0 to 8 and stored at $C051.

The Operation Manual describes the command but does not give the upper limit. `P9` is rejected.

Handler $EDA3 standard, $F063 CPR. **(No response)**

Q

`Q<n>`

Sets the service request mask. The digit is checked for 0 to 7 and stored at $C04F, where its three bits are separate enables: bit 0 internal errors, bit 1 input data errors, bit 2 settled at a new value. The range matches the eight combinations `Q0` to `Q7` in the Operation Manual.

Handler $ED47 standard, $EFF0 CPR. **(No response)**

R

`R`

`RM<n>`

`RL`

Bare `R` is the step-control right arrow.

`RM<n>` recalls user memory `<n>`, where `<n>` is 0 to 9, into the present value. Before overwriting the present value it swaps it into the last-value buffer at $C1CA, so `RL` can undo the recall.

`RL` recalls the last entered value by swapping the present value with the last-value buffer. It is the bus equivalent of the RCL LAST key, and is refused while a calibration or an entry is in progress.

The handler looks at the character after `R` without consuming it, so an `R` followed by anything other than `M` or `L` is the arrow.

Handler $ECE9 standard, $EF92 CPR. **(Bare R documented; RM and RL not documented)**

S

`SM<n>`

Stores the present value in user memory `<n>`, where `<n>` is 0 to 9. The eight bytes at $C069 are copied to $C35C + 8 x `<n>`, the user block checksum is recalculated, and the whole non-volatile image is written to NOVRAM. It is the bus equivalent of STO MEM followed by a digit.

`S` is not a command on its own. The handler requires `M` next and treats anything else as an undecodable command.

Handler $ED25 standard, $EFCE CPR. **(Not documented)**

Caution



Every `SM<n>` starts a NOVRAM store cycle, and these devices survive a limited number of store cycles. Do not send `SM<n>` in a loop.

T

`T0`

`T1`

`T0` selects four-wire operation, the power-up condition. `T1` selects two-wire compensation. The digit is checked for 0 or 1 and stored at $C052.

Handler $ED8C standard, $F04C CPR. **(No response)**

U

`U`

Step-control up arrow: increments the active digit.

Handler $ECA5 standard, $EF4E CPR. **(No response)**

Digits and Decimal Point

`0` **to** `9`

`.`

A digit or a decimal point starts numeric entry. The parser then runs its own loop, accepting digits and one decimal point, until it meets a character it does not accept. That character is pushed back and the dispatch table carries on from it.

An exponent is introduced by `E` or `e` and may be signed. Inside the exponent `+` and `-` are significant; at the top level of a command line they are ignored.

The Operation Manual says digits beyond seven are ignored. That limit comes from how the value is stored, not from the parser, which does not count digits.

Handlers: digits $EDE8 and decimal point $EDCF standard; digits $F0A8 and decimal point $F08F CPR. **(No response)**

8

Non-Volatile Data

NOVRAM Organisation

IC522 and IC523 are 256 x 4 non-volatile RAMs (NCR 52212 in the instrument examined). The Service Manual says IC522 holds the calibration data and IC523 the user data, and that the rear calibration keyswitch controls IC522's write enable while the processor controls IC523's. The firmware shows how the two devices share one address range.

Both devices start at $A000. IC522 supplies data bits 0 to 3 and IC523 bits 4 to 7, so each address holds one nibble of calibration data and one nibble of user data. The firmware never treats an $A000 location as a whole byte; it always packs and unpacks nibbles.

Note



Because the two blocks are interleaved by nibble, not separated by address, the calibration keyswitch can protect the calibration data while the user memories stay writable, with no address decoding to tell them apart. The Service Manual does not spell this out.

Block Layout and Checksums

RAM holds a copy of two blocks. Each has its own 16-bit checksum, stored straight after the data it covers.

Table 8-1. Non-Volatile Blocks

Block	RAM data	Data bytes	Checksum at	NOVRAM nibbles
Calibration	$C300 - $C357	88	$C358 - $C359	Low (IC522), $A000 - $A0B3
User	$C35A - $C3AB	82	$C3AC - $C3AD	High (IC523), $A000 - $A0A7

The user block starts with two bytes of configuration at $C35A, including the IEEE-488 address, followed by ten user memories of eight bytes each from $C35C to $C3AB. The internal structure of the 88-byte calibration block was not determined; the store routine writes it as one run of bytes.

Each checksum is the 16-bit sum of the range it covers, formed by adding successive words with `ADDD ,X++` and discarding any carry out of bit 15.

Store and Recall

Each direction is one routine, at the same address in both builds. The store routine at $E0FA recomputes both checksums and writes them into RAM. It then copies the blocks out a nibble at a time, low nibble of each byte first: the 90 bytes of calibration data and checksum fill the low nibbles of $A000 to $A0B3, and the 84 bytes of user data and checksum fill the high nibbles of $A000 to $A0A7.

Its last act is a read of $2000. That read is not a data access: the address strobes the NOVRAMs' store cycle, which copies their internal RAM into the non-volatile array.

The recall routine at $E165 reads $4000 to copy the array back into the devices' internal RAM, waits, rebuilds both blocks from their nibbles and checks both checksums.

Table 8-2. NOVRAM Control Strobes

Address	Access	Effect
$2000	Read	STORE: copy the internal RAM into the non-volatile array
$4000	Read	RECALL: copy the non-volatile array into the internal RAM

Caution



Non-volatile RAMs of this type survive a limited number of store cycles. The firmware runs a store every time a user memory is written, so a program that sends `SM<n>` in a loop will use up that endurance quickly.

Corruption Behaviour

Recall checks the calibration checksum first. If it fails, the firmware loads a default calibration block from ROM and shows `CAL DATA BAD`. If the user checksum fails, it shows `MEMORY DATA BAD`. Both messages are in Table 3-2 of the Operation Manual; what happens behind them is not.

A calibration checksum failure therefore does more than report a fault. The instrument goes on running with the ROM default constants, so its output is no longer calibrated, not merely suspect.

9

Calibration and Integrity Checks

The Calibration Step Counter

The calibration sequence is driven by one variable, the byte at $C012. It is zero whenever the instrument is not calibrating. Calibration starts when the main loop sees the rear keyswitch on (bit 6 of $C017 clear); the counter is then set to one, and each accepted step adds one. The sequence is complete when the counter reaches ten, which the code tests directly:

```
LDA   <$12          ; calibration step
CMPA  #$0A          ; ten steps in the sequence
LBCC  done          ; finished: store and exit
```

Much of the firmware reads this byte. The `N` command does nothing unless it is non-zero, the reset path clears it, and the display code uses it to decide whether to show calibration prompts.

A step can be passed without entering a value. The code checks whether any digits were entered and, if none were, moves on without storing anything. Section 6.3.4 of the Operation Manual describes this: a step may be skipped by pressing a unit key without adjusting the output.

Driving Calibration Remotely

The Operation Manual presents calibration as a front-panel procedure. Section 6.3.4 describes key presses, and nothing in Section 4 suggests the sequence can be driven over the bus. It can.

The `N` command reaches the same code as the front-panel unit keys: key 11 of the key dispatch table and the `N` entry of the command dispatch table both lead to $F30E in the standard build ($F5EF CPR). A controller can therefore send a value followed by `N` for each step, just as an operator keys in a value and presses a unit key.

Caution



Calibration cannot start unless the rear calibration keyswitch is on. Once it is on, a controller can complete a whole calibration with no one at the front panel, and the last step writes the result to NOVRAM. An automated test system that sends stray `N` characters to a 2724A with its calibration switch on can destroy the calibration without any warning on the display.

At the end of the sequence the firmware shows `TURN CAL SW OFF` and waits, testing bit 6 of $C017, until the keyswitch is turned off. It then jumps to the reset entry point.

The Banner Integrity Check

The firmware holds its model and version banner in more than one place and checks the copies against each other. In the standard build the check compares the fourteen bytes at $F9BE, the banner shown at power-on, with a second copy at $F766:

```
LDS   #$C23F          ; reset the stack
LEAX  549,PCR         ; $F9BE: displayed banner
LEAY  -54,PCR         ; $F766: reference copy
LDB   #$0E            ; fourteen bytes
LDA   ,X+
CMPA  ,Y+
BNE   fail            ; copies differ
DECB
BNE   loop
```

If the copies differ, the firmware shows a fifteen-character message that decodes to `FUCK FUCK FUCK`. A second check, at the start of calibration, can show the same message. It is stored at $E405 and $F775 in the standard ROM and at $E53F and $FAE1 in the CPR ROM.

This is a tamper check, not a fault indication. The banner tells a standard instrument from a CPR one, and a plausible motive is to make a standard ROM that has been patched to claim CPR capability give itself away. Whatever the intent, for a technician the message means the ROM has been altered, not that the hardware is faulty.

Note



The banner reads `2724A V-FB` in the standard build and `2724CPR V-FB` in the CPR build. The manuals quote it as the second power-on display but do not explain it.

10

Firmware Differences

Common Regions

The two builds are not one program with a patch. Only 1247 of the 8192 bytes (15.2%) are identical at the same address, and the CPR build's code is shifted throughout. What the builds share is a few library regions that sit at the same addresses in both.

Table 10-1. Regions Identical in Both Builds

Range	Bytes	Contents
$E002 - $E0A8	167	ROM checksum word, character table at $E058, first display messages
$E0DE - $E14D	112	NOVRAM checksum and store routines
$E16E - $E1B8	75	NOVRAM recall routine
$E1BB - $E201	71	Calibration default block loader
$FE71 - $FFF1	385	Low-level utilities, delay routines and the vector table

Because the NOVRAM routines are identical and at the same addresses, both builds store their non-volatile data in the same way. A calibration block from a standard instrument is laid out in NOVRAM exactly as one from a CPR instrument, though its contents need not mean the same thing.

Functional Differences

The CPR build has two extra key handlers, for the CARD PNT and PPM DEV keys that the Operation Manual describes for the option (Chapter 3), and one extra command character, `C`. Section 4.4 of the Operation Manual documents `C` as available on CPR units only, and the firmware agrees: the standard build has no `C` entry, so `C1` sent to a standard instrument is an undecodable-command error.

Table 10-2. Differences Visible in the Firmware

Feature	Standard	CPR
Command characters	23	24
Extra command	-	`C0` / `C1`: active or CPR mode
Front-panel key handlers	15	17
Banner text	`2724A V-FB`	`2724CPR V-FB`
Cardinal-point calibration prompts	None	Six, one for each cardinal point from 100 ohms to 10 Mohms, such as `CPR  1Ko/ CAL`
Command dispatch table	$EEC8	$F188
Key dispatch table	$F0C1	$F385
Key matrix table	$F43E	$F79C
Display routine	$E9E6	$EC55
Reset entry	$F9D8	$FD51

11

Verification Record

Checks Performed

The analysis was made from the ROM images alone. Since then the CPR image has been run in a 2724A-CPR, which shows that the image is complete and correct but does not test the undocumented behaviour. Table 11-1 lists each check and how it was made.

Table 11-1. Checks Performed

Check	Method	Result
Images are complete and uncorrupted	Firmware's own ROM test recomputed over both images	Both match the word at $E000
Load address is $E000	All eight vectors decoded and tested for range	7 of 7 live vectors point into ROM in both builds
Processor is a 6809	Parts list, 6809-only instructions and the FIRQ vector	Consistent
Disassembler is correct	68 hand-verified encodings covering every addressing mode	68 of 68 pass
Disassembly is complete	Source files reassembled and compared with the images	Both images rebuilt byte for byte
Command tables are complete	Tables walked to their zero terminators	23 standard, 24 CPR
Message decoding is correct	Record length tried at four candidate values	15 bytes; every record forms centred text
Documented commands agree with firmware	Argument ranges compared with Sections 4.4 to 4.6 of the Operation Manual	All documented ranges agree
Greeting strap decoding	Index arithmetic worked for all four strap combinations and compared with note 4 of drawing 2724-071	4 of 4 rows reproduced exactly
CPR image runs on the instrument	Programmed into an ST M27C64A-15F1 and fitted at IC519 of the author's 2724A-CPR	Boots normally; power-on test reports `CAL DATA OK`

The checksum result carries the most weight. Reproducing an 8K checksum exactly, with an algorithm read from the code, shows that the images are intact, that the load address is right and that at least the reset path is disassembled correctly. It says nothing about whether any interpretation of the code is right.

Not Verified

The following are inferences from the code. The document states them as fact only where the code leaves no reasonable alternative. Each could be settled quickly on the bench.

- That `SM<n>`, `RM<n>` and `RL` behave over the bus as described. The handlers are shared with the front-panel keys and the argument checks are clear, but no command has yet been sent to an instrument.
- That `N` advances calibration. The evidence is a shared handler address and a step counter compared with ten. The full ten-step sequence was not traced.
- The meaning of the 88 calibration bytes at $C300. Their extent and checksum are certain; their internal structure was not determined.
- That $2000 and $4000 are the NOVRAM store and recall strobes. This follows from the access pattern and the device type, and the CPR instrument recalls its calibration correctly at power-on, but no signal has been observed.
- That a 16-bit access at $03FF, $07FF or $13FF reaches two separate 8-bit devices (Chapter 2).
- The source of bits 0 and 1 of $C017, and which of K3 and K4 is on bit 4 (Chapter 3). The keyboard input buffer on drawing 2724-070 sheet 5 would settle both.
- The IEEE-488 controller type. The register layout fits a conventional talker/listener controller, but the parts list for board 2724-602 does not name the device.

How to Reproduce

The analysis rests on the two image files and a 6809 disassembler. The disassembly is also available as source files, 2724A_STD.asm and 2724A_CPR.asm, which reassemble to both images byte for byte.

The checksum test is the most useful single check. Sum every 16-bit big-endian word from offset 2 to the end of an 8192-byte image, modulo 65536, and compare the result with the word at offset 0:

```
s = 0
for a in range(2, 8192, 2):
    s = (s + (rom[a] << 8 | rom[a+1])) & 0xFFFF
assert s == (rom[0] << 8 | rom[1])
```

To find the command table in a similar build, find the dispatch loop by its signature, `CMPA ,Y+` followed by `LEAY 3,Y` and `TST ,Y`, and read the program-counter-relative offset in the `LEAY` before it. That offset points at the table.